eudamed2firstbase — GUI-Modi 0–5

Was jeder Knopf in der GUI tut, wann er der richtige ist und welche Stolperfallen er hat.

Document version: 2026-05-03 · Tool version: v1.0.47+ · Source of truth: `src/gui.rs`, `src/main.rs`, `src/download.rs`

1. Übersicht

Die GUI hat sechs nummerierte Knöpfe (Mode 0–5). Jede Pipeline schreibt ihren Modus in die erste Logzeile (`eudamed2firstbase v<version> — mode: <name>`) und in die `version`-Spalte der `push_session`-Tabelle, damit man hinterher genau weiss, welcher Knopf was ausgelöst hat.

Mode	Knopfbeschriftung	Eingabe	Pipeline
0	DL+Push <target>	SRN-Liste	Download → Convert → Push
1	Convert & Push (all)	—	Convert → Push (alles in `firstbase_json/`)
2	Convert & Push SRNs	SRN-Liste	Convert → Push (gefiltert auf SRN-UUIDs)
3	Repush failed	—	Restore aus `processed/` → Push (nur abgewiesene UUIDs)
4	Repush SRN	SRN-Liste	Restore aus `processed/` → Push (SRN-UUIDs)
5	Reconvert + Repush SRN	SRN-Liste	Reconvert aus `eudamed_json/detail/` → Push (SRN-UUIDs)

> **Erste Logzeile lesen.** Jeder Pipeline-Lauf beginnt mit `eudamed2firstbase v<version> — mode: <name>`. Wenn «Test» oder «Production» nicht das ist, was du erwartet hast, sofort abbrechen.

2. Mode 0 — DL+Push

0 DL+Push <target>

ABLAUF

1. **Download** — pro SRN: paginiertes Listing aus EUDAMED holen, in `listing_cache` schreiben, sofort gegen `udi_versions` prüfen (*new*, *udi-bumped*, *budi-bumped*, *same*). Nur Devices mit Versions-Bump oder ohne lokalen Eintrag werden detailmässig nachgeladen.
2. **Convert** — jedes Detail-JSON wird durch `transform_detail` in Firstbase-JSON umgewandelt. SHA256-Hash-Skip: unveränderte Devices werden übersprungen, ausser weder `firstbase_json/<uuid>.json` noch `firstbase_json/processed/<uuid>.json` existiert (Disk-Fallback seit v1.0.41).
3. **Push** — `Live/CreateMany` in 100er-Batches, Polling, dann `AddMany` auf den Recipient-GLN. Akzeptierte Files wandern nach `firstbase_json/processed/`.

WANN VERWENDEN

- Erstbefüllung einer SRN-Liste.
- Routine-Update wenn EUDAMED neue/aktualisierte Devices ankündigt.
- Wenn du nicht weisst, was sich seit dem letzten Lauf geändert hat — Mode 0 macht das Richtige.

WANN *NICHT* VERWENDEN

- Wenn du nur einen **Konverter-Fix** in Firstbase landen lassen willst (z.B. v1.0.46 SPP-Fix). EUDAMED-Daten sind unverändert → Download skipped, Hash matcht → Convert skipped. Nimm dafür Mode 5.
- Wenn du nur abgewiesene Devices nochmal probieren willst — nimm Mode 3.

FRISCHE DATEN AUS EUDAMED?

Nur für Devices mit Versions-Bump im Listing oder ohne `udi_versions`-Eintrag. Devices, deren Listing-`versionNumber` identisch zum DB-Stand ist, werden **nicht** neu heruntergeladen.

3. Mode 1 — Convert & Push (all)

1 Convert & Push (all)

ABLAUF

1. kein Download
2. Convert — alle Detail-JSONs in `eudamed_json/detail/` werden durch den Konverter geschickt (mit Hash-Skip).
3. Push — alles, was nach dem Convert in `firstbase_json/` liegt, wird gepusht.

WANN VERWENDEN

- Du hast eine grössere Detail-JSON-Sammlung lokal und willst «den ganzen Bestand» in Firstbase haben.
- Nach einem Konverter-Fix, der den ganzen Bestand betreffen kann (vorsichtig — siehe unten).

WICHTIGE EINSCHRÄNKUNG

Der Hash-Skip greift hier voll. Wenn ein Device in `processed/` liegt und der SHA256 unverändert ist, wird es **nicht** neu konvertiert — selbst dann nicht, wenn ein Konverter-Fix neue Felder erzeugen würde. Für gezielte Re-Konvertierung einer SRN-Untermenge ist Mode 5 die richtige Wahl.

4. Mode 2 — Convert & Push SRNs

2 Convert & Push SRNs

ABLAUF

1. kein Download
2. Convert — nur Detail-JSONs, deren UUIDs in `listing_cache` einer der gewählten SRNs zugeordnet sind.
3. Push — resultierende Files in `firstbase_json/`.

WANN VERWENDEN

- Wenn du **einen Hersteller** komplett neu aufbauen willst, ohne den ganzen Bestand anzufassen.
- Wenn der EUDAMED-Download von dieser SRN sicher schon mal gelaufen ist und die Detail-JSONs lokal liegen.

VORSICHT

Hash-Skip greift wie in Mode 1. Wenn du nach einem Konverter-Fix saubere Re-Konvertierung willst, nimm Mode 5.

5. Mode 3 — Repush failed

3 Repush failed

ABLAUF

1. Liest abgewiesene UUIDs aus dem `push_log` (Status REJECTED).
2. Stellt zugehörige `<uuid>.json` aus `firstbase_json/processed/` nach `firstbase_json/` wieder her.
3. **Push** nur dieser UUIDs.

WANN VERWENDEN

- Nach einem Push, der teilweise gescheitert ist (z.B. wegen 429-Throttling, transienter Validierungsfehler nach GS1-Regeländerung).
- Wenn du sehen willst, ob ein neuer Konverter-Build die alten Errors fixt — aber **Achtung**: Mode 3 nimmt das alte `processed/`-File, nicht die frisch konvertierte Version. Für letzteres: Mode 5.

WAS NICHT PASSIERT

- Kein Download.
- Keine Re-Konvertierung — das exakt gleiche JSON wird nochmal hochgeladen.

6. Mode 4 — Repush SRN

4 Repush SRN

ABLAUF

1. Liest aus `listing_cache` alle UUIDs zu den gewählten SRNs.
2. Stellt `<uuid>.json` aus `firstbase_json/processed/` nach `firstbase_json/` wieder her.
3. **Push** nur dieser UUIDs — bypassed `udi_versions` "unchanged"-Skip.

WANN VERWENDEN

- Wenn ein Hersteller in Firstbase «verloren» aussieht (z.B. weil ein vorheriger Push abgebrochen ist) und du den exakten letzten konvertierten Stand nochmal hochlädst.

WAS NICHT PASSIERT

- Kein Download, keine Re-Konvertierung. Du pushst, was beim letzten erfolgreichen Konvertierungslauf entstanden ist.

7. Mode 5 — Reconvert + Repush SRN

5 Reconvert + Repush SRN

ABLAUF

1. UUIDs aus `listing_cache` für die gewählten SRNs holen.
2. **Reconvert** — `transform_detail` auf `eudamed_json/detail/<uuid>.json` mit der *aktuellen* Konverter-Logik. Schreibt frisches `firstbase_json/<uuid>.json`.
3. **Fallback nur für UUIDs ohne Detail-File**: Restore aus `firstbase_json/processed/`. Logzeile: `WARNING: N UUID(s) have no eudamed_json/detail/<uuid>.json — will fall back to firstbase_json/processed/` (seit v1.0.46).
4. **Push** der frischen Files.

WANN VERWENDEN

- Nach einem Konverter-Fix, der einen bekannten SRN-Bereich betrifft (z.B. v1.0.45 ContactType-Fix, v1.0.46 SPP-Fix, v1.0.43 DescriptionShort).
- Wenn die EUDAMED-Daten sich nicht geändert haben, aber der neue Code andere/zusätzliche Felder erzeugt.

> **Stolperfalle.** Wenn die Detail-JSONs lokal fehlen, fällt Mode 5 stillschweigend (naja, mit der Warning-Zeile) auf das alte `processed/`-Material zurück — also auf die Konverter-Version, die den Fix noch nicht enthielt. Wenn die Warning erscheint: erst Mode 0 für die SRNs laufen lassen, damit die Detail-JSONs lokal sind, dann Mode 5.

8. Entscheidungshilfe — welcher Mode?

Situation	Mode
Neuer SRN, noch nie gepusht	0
Routine: was ist seit gestern in EUDAMED neu?	0
Konverter-Fix, EUDAMED-Daten unverändert, SRN-Liste bekannt	5 (ggf. 0 voraus)
Letzter Push hatte Reject-Errors, GS1-Regel hat sich geändert	5 (mit neuem Build) oder 3 (gleicher Stand)
Lokaler Bestand soll komplett raus	1 (Vorsicht: Hash-Skip)
Ein Hersteller ist in Firstbase verschwunden	4
Nur die abgelehnten UUIDs nochmal probieren	3

9. Was alle Modi gemeinsam haben

- **Push-Target** wird im Push-Target-Selektor festgelegt (Firstbase Test / Production oder Swissdamed). Das Banner im HTML-Logreport zeigt klar an, welche Umgebung verwendet wurde — rot bei Production, blau bei Test.

- **HTML-Push-Log** wird nach `log/firstbase_test/` bzw. `log/firstbase_prod/` geschrieben (seit v1.0.39 environment-getrennt). Enthält Errorzusammenfassung, abgewiesene Devices mit Errorcodes & Attributnamen, akzeptierte Liste und die rohe API-Response.
- **Pull-Push-DB**: jeder Push-Lauf schreibt eine Zeile in `push_session` mit Mode-Name, Environment und Counts. Per-UUID-Resultate landen in `push_log`.
- **Akzeptierte Files** wandern nach `firstbase_json/processed/`; abgewiesene bleiben in `firstbase_json/` liegen, damit Mode 3 sie wiederfindet.
- **WhatsApp-Reporting**: in der GUI gibt es zwei Knöpfe — rot «Send latest Prod log» und blau «Send latest Test log» — die jeweils das neueste HTML-Log aus dem entsprechenden Subdir senden.

---

10. Häufige Fragen

Wenn ich Mode 0 (DL+Convert+Push) drücke, holt das alle Datensätze frisch aus EUDAMED?

Nein. Mode 0 holt das paginierte *Listing* für jede SRN (das ist günstig — ein API-Call pro 20 Devices), schreibt es in `listing_cache` und prüft dann pro UUID gegen `udi_versions`. Nur Devices mit Versions-Bump oder ohne lokalen Eintrag werden detailmässig nachgeladen. Wenn EUDAMED nichts bumped, wird auch nichts neu heruntergeladen.

Warum sehe ich nach einem Konverter-Fix die neuen Felder nicht in Firstbase?

Wahrscheinlich Hash-Skip oder `processed/`-Restore. Der Konverter prüft pro UUID einen SHA256-Hash der EUDAMED-Daten — wenn der unverändert ist, wird übersprungen. Mode 5 umgeht das, weil er `transform_detail` direkt aufruft, ohne den Change-Detector zu konsultieren.

Was, wenn Mode 5 die Warning «will fall back to firstbase_json/processed/» loggt?

Bedeutet: einige UUIDs der gewählten SRNs haben kein lokales Detail-JSON, daher kann der Konverter sie nicht neu erzeugen. Mode 5 verwendet dann das alte `processed/`-File — also die Konverter-Version vor dem Fix. Lösung: zuerst Mode 0 für diese SRNs (lädt das Detail-JSON), dann Mode 5.

Warum sind das 6 Knöpfe und nicht 5?

Modes sind 0-indexiert: 0, 1, 2, 3, 4, 5 — macht sechs. Die Nummer steht seit v1.0.47 vorne in der Beschriftung, damit sich Knopf, Logzeile und `push_session.version` direkt korrelieren lassen.

Wie unterscheide ich im Log nachträglich, welcher Mode gelaufen ist?

Erste Logzeile jedes Pipeline-Laufs: `eudamed2firstbase v<version> — mode: <name>`. Zusätzlich steht der Mode-Name in der `push_session`-Tabelle der SQLite-DB.

Zeno Davatz
ywesee GmbH · ywesee.com